

Portuguese Song Search System

João Pedro Dias Fernandes
up202108044@fe.up.pt
Faculty Of Engineering of University of Porto
Porto, Porto, Portugal

Igor Monteiro de Andrade
up202108674@fe.up.pt
Faculty Of Engineering of University of Porto
Porto, Porto, Portugal

João Brandão Alves
up202108670@fe.up.pt
Faculty Of Engineering of University of Porto
Porto, Porto, Portugal

Ema Maria Monteiro Martins
up202402794@fe.up.pt
Faculty Of Engineering of University of Porto
Porto, Porto, Portugal

Abstract

With the constant evolution of the internet, the amount of information available on the internet are getting even bigger. With that in mind, to address those big amounts of data, it is important to have specialized search systems to address those areas.

We are focusing on creating a system to search and retrieve information about music in Portuguese that was produced in 2016, as it is one of the biggest entertainment industries in Portugal and, usually, the lyrics are misunderstood by the listeners, where a search system will help in that question.

This article contains all the needed steps to develop a robust search engine to address that area. That contains collecting data from Kaggle and Last.FM, clean, prepare, and characterize data, then we index using schemas on Solr, and then, on some prospective search tasks, we intend to query it and evaluate our results.

To reach the current state of the project many challenges were overcome in every and every part of the project, like the troubled join of the dataset with the Last.FM API, finding the correct search tasks for our system, the initial creation and subsequent improvement of the schemas and the test with using semantic search, but the result is a search engine that is proven useful to find the song you desire in the middle of an ocean that is growing every year.

CCS Concepts

• **Information systems** → Information Retrieval and Query Processing.

Keywords

Information retrieval, Data preparation, Portuguese, Music, 2016, Natural language processing, Text mining, Search system, Document indexing, Music reviews, Music Stats

ACM Reference Format:

João Pedro Dias Fernandes, João Brandão Alves, Igor Monteiro de Andrade, and Ema Maria Monteiro Martins. 2024. Portuguese Song Search System.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

FEUP, Porto, 19 December 2024,

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

In *Portuguese Song Search System*, December 19, 2024, Porto, Portugal. ACM, New York, NY, USA, 18 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

In today's world, the huge amount of digital information available around the web has made it necessary to have effective tools for searching and retrieving relevant data. Our project focuses on creating a search system centred on Portuguese music that was produced and released in 2016 — a year that was rich and diverse in musical output and cultural significance. The choice of language and year came from our feeling of nostalgia for a year that turned out to be full of achievements for our birth country. By collecting, and processing data related to this music dataset, indexing and also retrieve, we intend to provide a system that can easily search through lyrics, reviews, and other related documents like stories about how the songs were created. We also managed to perform some queries and evaluate them to test all the system.

This report details all the process of creating a robust search system, since the data collection phase, which englobes the extraction, preparation and characterization, followed by indexing of the same one, which leads to the retrieval, where we define some attributes of the queries. We also proceed to perform some evaluation on our system, to assure it can reach our ideal scenario of information needs.

This report is structured in the following sections: First, the data extraction, where we describe the initial dataset, the enrichment done, and the final clean + finalization. Secondly, the data preparation, where we describe the data pipeline, which includes data cleaning and restructuring. After that, we characterize the data in Data Characterization. We also prospect some search tasks that are expected to be fulfilled by our system. It is followed by the Data Indexing, where we focus on explaining all the processing made to data upon query time. After that, we debriefed a bit about the interface. We also explain all the retrieval preparation that will follow all the queries done in the Data Evaluation, where we address the prospective search tasks on our system and evaluate them, if they were addressed correctly or not. In the end, we conclude our work and also some images at the Annexes, mostly.

2 Data Extraction

Following a search for pertinent datasets, two datasets were found: one in CSV format, obtained from Kaggle, and the other in JSON format, gotten from an API called Last.FM, an API that specializes in extracting music statistics to certain users that can link their music

player accounts (example: *Spotify*, *Deezer*, *Apple Music*, *Soundcloud*, etc.). That is followed by a brief overview of the datasets that we chose.

2.1 Initial Dataset Characterization

Kaggle's initial dataset had a total of 9050 records of songs, which included attributes such as song title, artist, year of release, tag, number of views, language, and lyrics. This dataset, although no license is exactly attached to it, has a rating of 8.24 in *Kaggle*. The source of this dataset is the *Genius* dataset, where it was applied to a model of language recognition of lyrics. The *Last.FM*'s dataset had a way bigger total of records, as it holds statistics for way more songs than the ones that we are using (songs in Portuguese from 2016). *Last.FM* grants a limited but enough free license to use it, especially for students. By grouping both datasets, we added some attributes like the number of different listeners, the time play count of the songs, some text content about the song and the artist, an image of the album, and an image of the artist. As some of *Kaggle*'s dataset songs had no match in *Last.FM*'s dataset, we decided to remove those songs from the final dataset. With this, the final dataset's total number of records turned a little lower than the 9050 that we started with.

2.2 Dataset Enrichment

To upgrade the initial dataset, additional data points were gathered from external sources (*Last.FM*). This included fields such as the number of different listeners, total play counts, tags assigned by listeners, and images related to the song, artist, and album. After this, the dataset grew to include more complex attributes, such as information about the artist and the album. However, there were still some missing values, especially in multimedia fields.

2.3 Data Cleaning and Finalization

Data cleaning was performed to remove incomplete records and reduce noise, resulting in a final cleaned dataset of 7885 entries. This part involved eliminating duplicate records and filling in missing values where possible. The cleaned dataset contained the following key attributes:

- Song metadata: title, artist, year of release, and tags.
- Listener engagement metrics: play counts and different listeners from *Last.FM*.
- Language detection results and textual content like lyrics, artist descriptions, and song stories.
- Multimedia elements: album and artist images.

These final attributes will be used to support some better analyses.

3 Data Preparation

In this section, we are presenting our data pipeline, which explains the processes of both data collection and preparation. This process is a **must** for the project as it ensures that: each item has interesting information from different sources; uniform data; and clean information.

Primarily, we focused on reducing our dataset from *Kaggle* to only Portuguese songs from 2016, mainly to ensure the data was

within an order of magnitude that would make sense in the context of this project. The selection of language and year originated from our feeling of longing for a year that ended up being filled with accomplishments for our native land. We analyzed the dataset and noted that the reduced dataset had some null or "no-content" rows, but we decided to ignore it, as it didn't prejudice the primary focus of this project.

With our dataset reduced, we performed queries to the *Last.FM API*. Those queries were firstly using the **desiredComponent.trackInfo** method, but if no match is found, we appealed to the **desiredComponent.search** method and retrieved the best match. However, if, still with the search method, the *API* doesn't find any results, we simply assign the **null string**. From the previous *API*, we gathered such information as different listeners, play count, the history and content of that song, the resume of the artist, associated tags, the image of the artist and the album, and the album name. Note: **desiredComponent** can be a track, album, or artist.

After the data gathering from the *API*, and adding the new columns to the dataset, we performed some **data preparation**, to achieve a more consistent dataset. Firstly, we noticed that some songs haven't got any matches in the *Last.FM API* (around 12%). As it was not so a big percentage, adding the fact of the database already being in the limit of the expected size (around 9000), we decided to eliminate those songs from our database.

After that, we only have one relevant null entry (not considering *Last.FM* entries), which was the title of a song, simply solved by **manual research and input** (what's the title of the song with these lyrics?). To note that we **eliminated** some columns that are not useful for this project (*language* and *language_ft* - these columns are part of a study of language recognition that was done by the author of the dataset we used as a source; we also eliminated *year* and *id*). Also, some columns like *tags*, *content_artist*, *content_song*, and others with information that came from *Last.FM* were **renamed** to make clear what information is from the *API*. Finally, the double quotes were removed from the features column to make the data simpler.

The result of all that pipeline is the *dataset_finale_cleaned*, a dataset that, although not 100% complete, is clear, simple, and easy to work through.

In Figure 26, we have a diagram of the entire pipeline structure, with the proper phases indicated and with their proper description.

3.1 Conceptual Data Model

In this Figure 1, the **conceptual data model** of this dataset is shown. Each entity has its elements (those who were extracted from *Last.FM* were identified) and its relations. To note that some songs are considered **singles**, in other words, they are not part of an album, so we have a unique relationship for that case.

4 Data Characterization

The focus of this section is characterizing the document obtained by the pipeline. To do that, some libraries were used: *pandas*, *matplotlib*, *collections*, *nlk* and *wordcloud*, to generate some graphs, tables, and a word cloud.

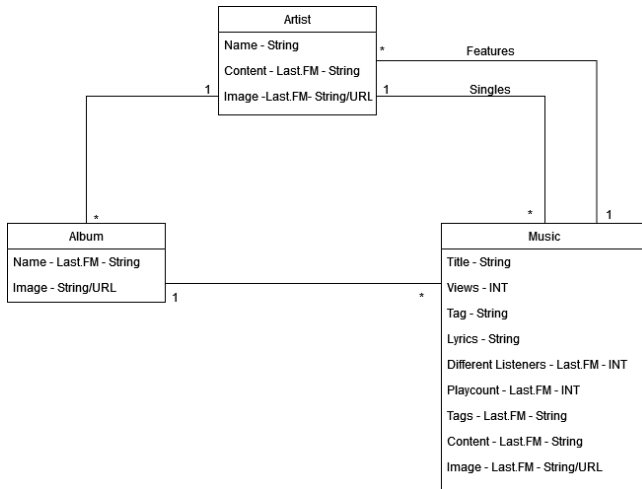


Figure 1: Conceptual Data Model.

4.1 Data Types

The dataset is composed of 7885 lines and 14 columns. Table ?? contains the description of the existing attributes present on the document.

4.2 Full Text Search

The word cloud in Figure 25 represents the main words present in the lyrics of the different songs. Some topics, such as *amor*, *vida*, *casa*, *mundo*, are present in the word cloud, which was expected since they are the main topics for a vast part of the Portuguese songs.

They are keys for the search tasks needed for the next stage, which is to search for information. In this case, looking for songs based on words present in lyrics is one of the searches that are possible to implement (Figure 3). It can be used, for example, to search for romantic songs. We can use words like "amor", "romance", "beijo", and "abraço" to give more weight to the songs that contain these words. This weight will be used in the process of ranking the retrieved results. Many other searches can be done because they have sufficient content to do so, like finding songs based on the song information (by the *lastfm_content_song* attribute, Figure 2), on the artist/band information (by the *lastfm_content_artist* attribute, Figure 4) and by the song's tags (by the *lastfm_tags* and tag attributes, Figure 5). For example, if the song information speaks about the feeling or the message of the music, it can be used in more abstract searches such as "happy songs" or "relax songs." Another example is that by searching the artist information, if an artist is Brazilian, the artist has more chance to have funk as a genre of music than a Portuguese one.

4.3 Tags distribution

Figure 6 shows the distribution of the tags. The most common tags are *rap*, *pop*, and *rock*. That result is expected once they are the most common styles that people listen to, not only in Portugal but all over the world.

The tags present in Figure 7 show that the most common Last.FM user tags also include *rap*, *pop*, and *rock*, which proves the importance of this kind of genre.

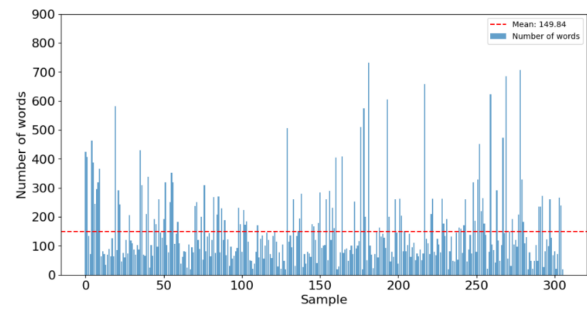


Figure 2: Number of words per song content.

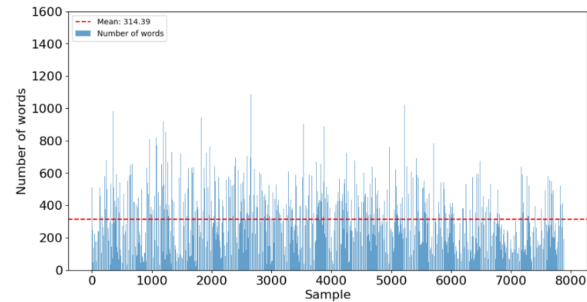


Figure 3: Number of words per lyric.

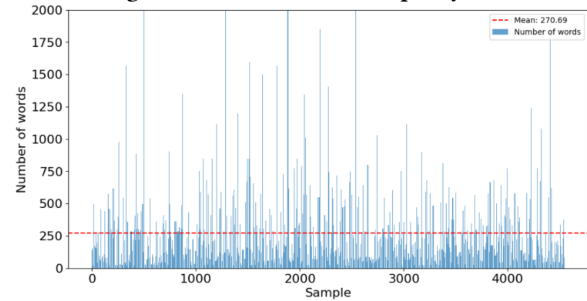


Figure 4: Number of words per artist content.

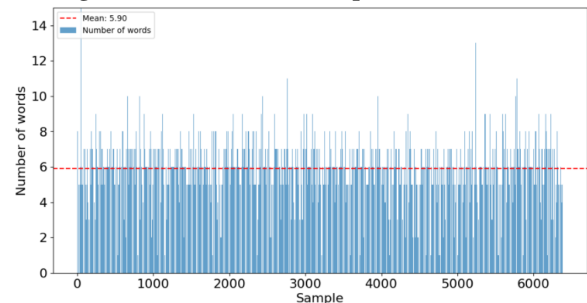


Figure 5: Number of words per Last.FM tag.

4.4 Singers/Bands distribution

Figure 8 shows the ten most common singers/bands. In that year, as observed in the table, the singer that had more singles in the dataset was the *MC Igu*.

Even though *MC Igu* was the artist with more songs, it wasn't the most listened to by *Last.FM* users. The Figure 9 *listeners_artist* shows that *Slim Boy* was the most listened to.

As represented in Table 1 and Table 2, the tags present in the songs of the artists with more songs and the artists more listened to in *Last.FM* matches the previously illustrated information, since the most common tags in both cases are rap.

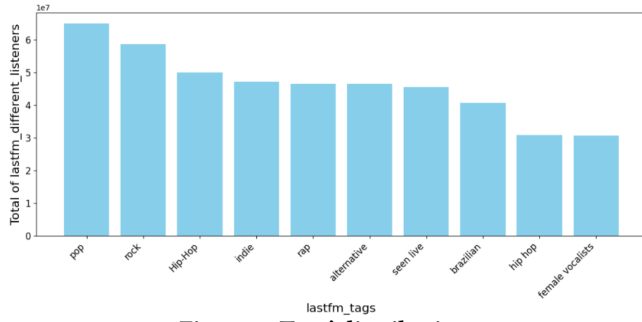


Figure 6: Tags' distribution.

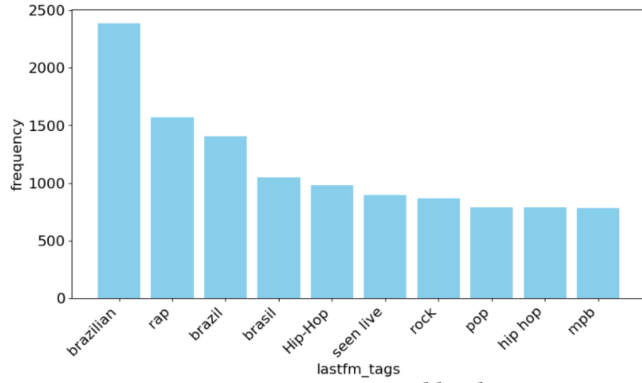


Figure 7: Top ten tags most used by the users.

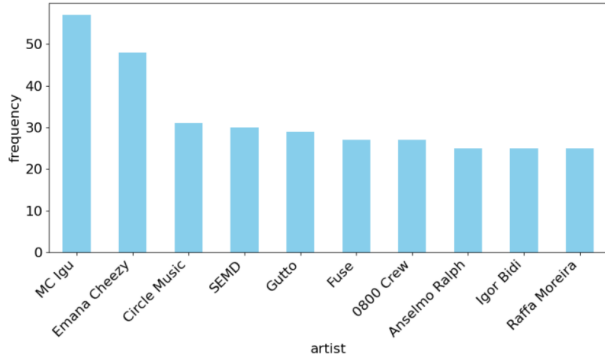


Figure 8: Top ten artists with more songs.

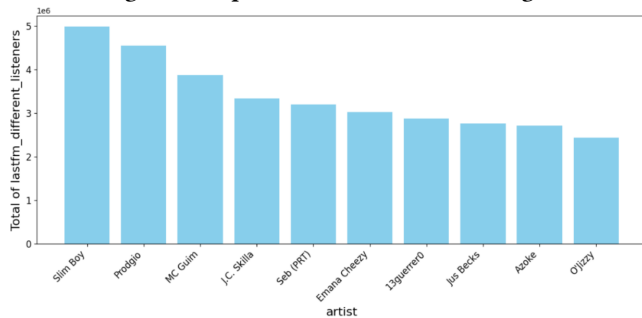


Figure 9: Top ten distribution of artists most listened to by Last.FM users.

5 Prospective Search Task

After this, through analysis of the data collected and the features of said data, it is possible to determine some possible search tasks the user may want to use:

Artist	Tags
MC Igu	rap
Emana Cheezy	rb, misc, rap
Circle Music	pop
SEMD	misc, rap
Gutto	rb
Fuse	rap
0800 Crew	rap
Anselmo Ralph	rb
Igor Bidi	rap
Raffa Moreira	rap

Table 1: Tags related to the songs of the artists with more songs

Artist	Tags
Slim Boy	rb, rap
Prodigio	rb,rap
MC Guim	pop,rap
J.C Skilla	rap
Seb (PRT)	rap
Emana Cheezy	rb, misc, rap
13guerrero()	rap
Jus Becks	rap
Azoke	rap
O'Jizzy	rap

Table 2: Tags related to the songs of the artists with more listeners in *Last.fm*

- Find songs with lyrics related to religion.
- Find songs whose artist has hit number 1 in the charts and is Portuguese.
- Find songs whose artist is female that talk about empowerment.
- Find songs that talk about reflection or love.
- Find pop songs that talk about the past.

In a resumed way, we want the users to be able to find any song with any knowledge they have about the song, even if it is only if the song is a *happy* or *sad* song, for example.

6 Data Indexing

Data indexing is a technique that enhances database performance by minimizing the amount of disk I/O (input/output) necessary to retrieve data. This technique is crucial to building an efficient information retrieval system. It is based on documents, which is a result of data restructure.

To help us in this phase, we will use Solr, an open-source search tool written in Java, which offers some reliable tools such as powerful data indexing and querying, supported by a full-text search feature.

In this section, we will present the document definition and the indexing process, which relies on the use of schemas, which we will detail later in this article.

6.1 Document Definition

The documents to be indexed are the result of the data pipeline previously referred, which was collected, prepared, and cleaned for this phase.

Each song comprises a title, an artist, features, views, and a tag, and it is also provided from Last.FM, a play count, the number of different listeners, and the content of both song and the artist. We have some long textual fields, such as the content of the artist and song and also lyrics, which will be useful for these further steps.

6.2 Indexing Process

Indexing a database can be quite troublesome if done ground-up. To help us, Solr provides a set of tokenizers and filters already defined and the possibility to define a schema with those features when uploading data to a core to help us in that task.

Tokenizers are classes that break down text into individual tokens, which are typically words or terms, in order to avoid problems with empty spaces or so what. Filters take those tokens and apply rules in order to modify, analyze, or enhance. In each field type in a schema, we can apply one tokenizer and various filters. Note that those filters and tokenizers can be applied in the indexing phase (when loading up the data) but also in the querying phase. In our tests, when applying the same tokenizer and filter(s) in both phases, it produced better results.

In our work, we focused on textual fields, preferably long ones. As our lyrics field (the focus field of our work) is in Portuguese, it was quite troublesome to find and apply some filters due to language limitations. That said, we used the following tokenizers and filters in our work:

- **StandardTokenizerFactory**: Breaks text into words or expressions to allow a simple rule applying by filters.
- **ASCIIFoldingFilterFactory**: Converts any non-ASCII characters to their ASCII equivalents.
- **LowerCaseFilterFactory**: Converts all uppercase letters to their lowercase version.
- **PortugueseStemFilterFactory**: Reduces Portuguese words to their root form. (Example: casinha -> cas).
- **SynonymGraphFilterFactory**: Expands tokens to include their synonyms. We also defined those synonyms in synonyms.txt as the project owner.
- **FlattenGraphFilterFactory**: Simplifies complex token graphs into a linear sequence for easier processing.

We used all of the above in the user-created-type boostedText and the first three ones in the shortText type, as it wasn't needed to apply such indexing to fields not relevant and/or fields with different language contexts (for example, artists' names). We used the SynonymGraphFilterFactory to allow accepting, in a query, synonyms or terms in the same context of a desired term (example: Jesus -> Deus, ser divino, igreja). It allows us to produce better results in our data context, as music searches by theme can be quite subjective, and maybe, for example, a religious song may not cite Jesus in its lyrics.

The structure of our schema is described below.

6.3 Indexing Improvements

After some previous evaluations, we discussed some potential improvements to the search system, as we believe that some results were potentially improbable.

Field	Type	Indexed?
lyrics	boostedText	yes
title	boostedText	yes
views	Int	no
artist	shortText	yes
tag	string	yes
features	shortText	yes
lastfm_different_listeners	Int	no
lastfm_playcount	Int	no
lastfm_tags	string	yes
lastfm_content_song	boostedText	yes
lastfm_content_artist	boostedText	yes
lastfm_image_artist	string	no
lastfm_image_album	string	no
album	string	yes

Table 3: Advanced schema fields type definition

About filters, the **PortugueseStemFilter** was removed because it possibly can cause some misleading in words that have the same stem. For example: *casa* and *casada* have the same stem.

Also in the filters context, we added the **StopwordsFilterFactory** to prevent some words that do not represent mostly any kind of meaning for the query, etc. And also some common words that are irrelevant.

Note that we also added some **synonyms** by requesting an online dictionary page (in this case, the website dicio.com.br) and used the BeautifulSoup library to extract information from the HTML page.

6.4 Semantic

In the context of music analysis (story and theme, for example), it is crucial to understand the semantic meaning of some words. They can fit different elements from a set of contexts that we need to be aware of, as we mostly expect to gather songs by their themes in this work.

To gather embeddings, which means converting documents into values that will be used for clustering, we used *neuralmind/bert-base-portuguese-cased*, an AI model specialized in Portuguese text, chosen due to the context of the project. We also did a few simple tests with the bigger version of this model, but, as the outputs seemed to have similar results, we decided to go for the simple model.

7 Interface

As this project intends to be a search system, designed thinking of the end user, it is crucial to guarantee a user interface that fulfils all our needs.

With that in mind, we developed a simple and basic interface that has a text input to introduce the query and also a dropdown box to select between a non-semantic or a semantic search, and when triggered the search button, it shows all the result documents and also the score associated, as this intends to be a prototype version.

All of this was made using React and an API using Flask in Python.

8 Retrieval

The retrieval involves using the pre-created information needs along with two different schemas: a simple schema with just default field types and an advanced schema that uses the filters mentioned in the last section. To achieve this, we used some query parameters, where some vary between simple and advanced and others stay the same. The parameters that stay the same independent of the schema are:

- **Query (q)** - Takes the most valuable words of the information need to perform the query;
- **Query Operator (q.op)** - Defines the default operator for query expressions (AND or OR);
- **Filter List (fl)** - Limits or adds to the information included in a query response.

These parameters were set as follows:

Parameter	Value
q	religião
q.op	AND
fl	*, score

Table 4: Common query parameters

The value from **q** is the one used for the first information need, used here as an example. Since **q** changes from query to query but not from schema to schema, the value of **q.op** was chosen to be AND, as, upon a quick inspection, it presented results better aligned with our vision of the desired outcome when compared to its counterpart OR, and finally the value of **fl** was changed just to give the ability to visualize the score of each one of the results, which was mainly used for finding out the best approach to get the desired results.

For the advanced schemas, we also used its parameters, with the intent to further optimize the search. In the simple search, one of the parameters was also used (**qf**) but without any of the boosts (lyrics, title, lastfm_content_artist tag artist).

The parameters used were:

- **Query Fields (qf)** - Takes a list of fields, each of which is assigned a boost factor to increase or decrease that particular field's importance in the query;
- **Phrase Fields (pf)** - Used to "boost" the score of documents in cases where all of the terms in the **q** parameter appear in close proximity;
- **Phrase Slop (ps)** - Defines the maximum number of positions one token can be moved in relation to another token in order to match a phrase specified in a query.

These parameters were set as follows:

Parameter	Value
qf	lyrics^ 4 title^ 2 lastfm_content_artist^ 3 tag^ 10 artist
pf	lyrics^ 6 lastfm_content_artist^ 6 title^ 10
ps	3

Table 5: Boosted schema parameters

Let's now explain the reasoning for these choices. The **qf** values were chosen based on our vision of the resulting output; if a music

genre is mentioned in the query, we want that to be taken into consideration more than any other thing, since we think that if a user searches for an exact music genre, he pretends to find only music of said type, therefore the 10 times boost. This is followed by lyrics and lastfm_content_artist, which are our open text fields and the main focus of our search system, which culminate in having the second highest boosts, with a slight edge to lyrics, which we deem a bit more important, which are then followed by the title and the artist in this order, which we consider important but not by a lot, resulting in these smaller boost values.

The **pf** were chosen based on similar criteria, with higher priority given to lyrics and lastfm_content_artist, as they were the main focus of our search, but this time with the addition of a 10 times boost to the title in case the word in the query matches in close proximity to the title, as we think if a user searches for a title in specific, it wants said song to be presented first. Finally, the **ps** value was chosen to be 3, as we think it is a reasonable value of distance between words to justify applying a boost to the result.

9 Evaluation

One of the most important steps in determining how good an information retrieval system is is to evaluate query performance. In this case, we examined a series of searches intended to extract pertinent results for a compilation of Portuguese music from a Solr-based search engine. As part of this assessment procedure, queries with various parameters—such as fields and boosting configurations—were run, and the recall, precision at 10 (**P@10**), precision at 30 (**P@30**), mean average precision (**MAP**), and average precision (**AvP**) metrics obtained from the outcomes were examined. To obtain these metrics, we used `trec_eval`.

- **Recall:** The percentage of pertinent documents that were recovered from the whole collection of pertinent documents. Fewer pertinent documents are missing when recall is high.
- **Precision:** The percentage of pertinent documents returned. The majority of recovered documents are helpful when accuracy is high.
- **Precision at 10 and 30 (P@10 and P@30):** A targeted precision statistic that assesses how relevant the top 10/30 documents that were retrieved are. For systems that prioritize rankings, this is helpful.
- **Average Precision (AvP):** Highlights the ranking order of pertinent things by calculating the average precision values at each time a relevant document is obtained.
- **Mean Average Precision (MAP):** The mean of the average precision of all the evaluation's queries. It offers a single metric that blends ranking and accuracy.

To do the evaluation, we retrieved 30 results per query, a value that we consider reasonable to have a vision about our system. For each of these results, we evaluated manually if the result was relevant or not. With that data, we calculated the precision-recall curves and the MAP. Even though we retrieved 30 results, we consider **P@10** instead of **P@30**, since normally the first 10 results are the ones that appear on the first page of a search system.

There are 4 schemas used to do this evaluation. The first two (simple and advanced) were the ones developed initially, and, with the results obtained, 2 more schemas were created, the improved

and the semantic, in an effort to enhance the results obtained in the first evaluation.

All the queries in the simple, advanced, and improved systems follow the same structure as exemplified below:

Parameter	Value
q	<query>
q.op	AND
fl	*, score
qf	lyrics^ 4 title^ 2 lastfm_content_artist^ 3 tag^ 10 artist
pf	lyrics^ 6 lastfm_content_artist^ 6 title^ 10
ps	3

Table 6: Parameters for queries in simple, advanced, and improved schema

There's only one thing that changes between the queries, and that is the q value. This parameter is defined as mentioned below:

Parameter	Value
q1	religião
q2	number 1 AND portuguese
q3	female AND poder
q4	reflexão OR amor
q4	pop AND passado

Table 7: Definition for q for all the queries in simple, advanced, and improved schema

The queries for the simple schema differ in that you only need to include the exponents for the fields in the qf parameter, while the pf and ps parameters are not required and should be removed.

The semantic schema has a different definition for the queries since they need to be generated based on vectors. To have the information for the query, we used a model to generate the vectors for each song and then for each query. With the provided information, we filled the query parameters, as illustrated below:

Parameter	Value
q	"!knn f=vector topK=30 [query vectors]"
fl	*, score
start	0
rows	30

Table 8: Parameters the semantic schema

The subsections below will present the results for the different queries in the different schemas and compare them. The top 10 relevant or non-relevant retrieved documents for each query are presented in Table 16, Table 17, Table 18, Table 19 and Table 20.

9.1 Query 1

Information Need: Songs with lyrics related to religion.

This query was chosen to explore how religious themes, a significant cultural element, are represented in Portuguese music and to assess the system's ability to retrieve songs with this specific focus.

Evaluation method: In this case, we check if the lyrics are talking about topics like Jesus or some kind of religious references.

We notice that if the music has `lastfm_tags` like `gospel`, it is a signal that it can be related to religion.

Results: Based on the $P@10$ results (Table 9), we can conclude that the advanced system performs better than the simple schema by a large margin. Following that we improved the advanced schema and created the improved schema, which had slightly better results than the advanced results in the top 10 results obtained, but the quality of the top 30 results got a little worse. In this query, the improved schema created a trade-off between the $P@10$ and the $P@30$. Looking for the $P@30$ and $P@10$ (Table 9), we can see that the simple system has a poor performance, while the advanced and improved systems have a trade-off between the $P@30$ and $P@10$. A semantic system was also created and tested after the simple and advanced schemas, and values show that the system did not retrieve relevant documents within the top 10 or 30 positions. The AvP (Table 9) demonstrates that the improved system has better general performance.

Rank Systems	Simple	Advanced	Improved	Semantic
P@10	0.5000	0.6000	0.7000	0.0000
P@30	0.3333	0.7333	0.6667	0.0333
AvP	0.5272	0.7317	0.7525	0.0417

Table 9: $P@10$, $P@30$, and AvP for query 1

These results of the evaluation show that there are differences in the performances of the four systems. Similarly, the precision-recall curves of the Simple System and the Advanced System exhibit different behaviors. In the Simple System, the precision drops drastically at low recall (Figure 10), while in the Advanced System (Figure ??), precision remains high for low recall, thus exhibiting better early retrieval capabilities.

The improved system outperforms both, as shown by its above $P@10$ and competitive $P@30$ results. In fact, the $P@10$ of 0.7000 already shows that most of its top 10 results will be relevant. Although slightly lower in value, its $P@30$ of 0.6667, against the Advanced System's 0.7333, suggests that while the Advanced System does exceptionally well on a broader set of results returned, the Improved System focuses its precision more on the items that ranked higher.

On this query, there is noticeable performance from the different systems judging from the AvP obtained. The improved system provides an AvP of 0.7525. Therefore, this system better represented the number of documents while remaining precise over varying recalls. The Advanced System got very close with 0.7317, which clearly evidenced an excellent performance but was very closely lower than that exhibited by the Improved System. The Simple System has a moderate AvP of 0.5272, indicating that while it retrieves relevant documents, its precision is less consistent. On the other hand, the semantic system has a very low AvP of 0.0417, which indicates high difficulties in identifying relevant documents for this query. The improved and advanced systems are clearly ahead of the rest regarding retrieval effectiveness.

Overall, the improved system achieves an improved balance between precision and recall, making it superior in terms of retrieving relevant items at the cost of minimal retrieval of irrelevant results. Other future evaluations should also account for the average precision AvP of the improved system to provide a fairer comparison.

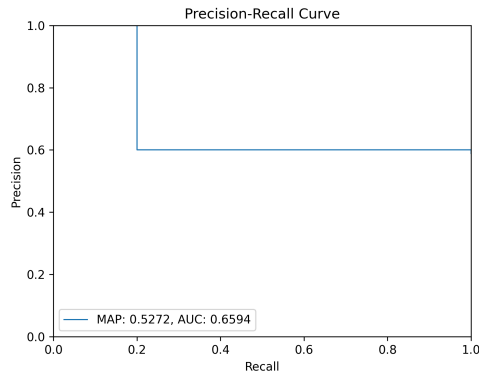


Figure 10: Precision-recall curve for query 1 in a simple system

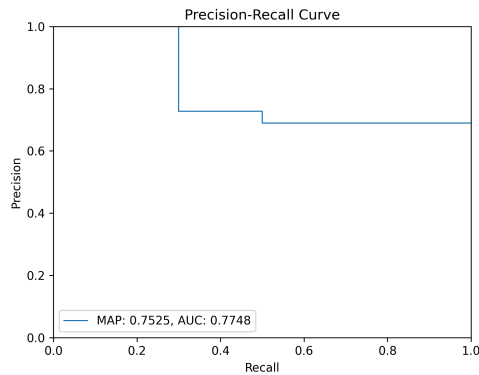


Figure 11: Precision-recall curve for query 1 in the improved system

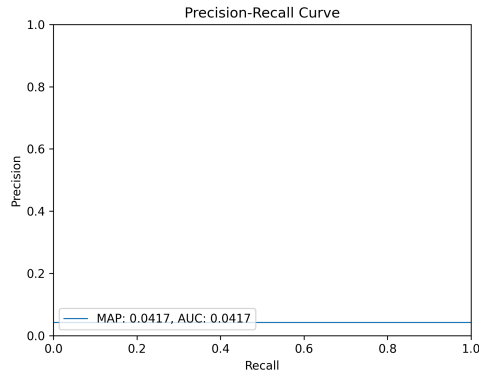


Figure 12: Precision-recall curve for query 1 in the semantic system

9.2 Query 2

Information Need: Songs whose artist has hit number 1 in the charts and is Portuguese.

This query was selected to identify top-charting Portuguese artists, showcasing the system's capability to retrieve prominent figures in the national music scene.

Evaluation method: We determined that to be number 1, the singer needed to have more than 10 million views. To check if the

artist is Portuguese, we saw the `lastfm_tags` to understand if they have some references to Portugal and also the `lastfm_content_artist`.

Results: The $P@10$ (Table 10) demonstrates that the advanced system is completely effective, while the simple system has a considerably lower performance. Based on the $P@30$ (Table 10), the advanced system isn't more completely effective, even though it is much better than the simple system. Comparing the results obtained from the improved system with the results of the advanced system, we concluded that both systems retrieved the exact same results, meaning that the changes made to the improved system really had no effect on this query, as the advanced system already returned really good results. The results for the semantic schema show that the system doesn't retrieve relevant results within the top 10 and top 30 ranks. AvP (Table 10) confirms that the advanced system maintains good results through all the retrieved processes, while the simple system maintains very low and inconsistent performance.

Rank System	Simple	Advanced	Improved	Semantic
P@10	0.3000	1.0000	1.0000	0.0000
P@30	0.1333	0.6667	0.6667	0.0333
AvP	0.3000	1.0000	1.0000	0.0417

Table 10: P@10, P@30, AvP for query 2

In this example, you can see perfect performance as shown by the precision-recall curve (MAP and AUC = 1.0). This means that the model has ideal precision (all positive predictions are correct) at all levels of recall. This is a perfect scenario where there are no false positives and negatives (Figure ??). This curve has a lower MAP and AUC than the previous one (Figure 13); therefore, the latter model is suspected to be more accurate and robust compared to the former.

The results for the AvP for this query are that both Advanced and Improved Systems achieved perfect scores of 1.0000, while the Simple System is way lower at 0.3000, and the Semantic System, again far behind, is very low at 0.0417, showing poor effectiveness.

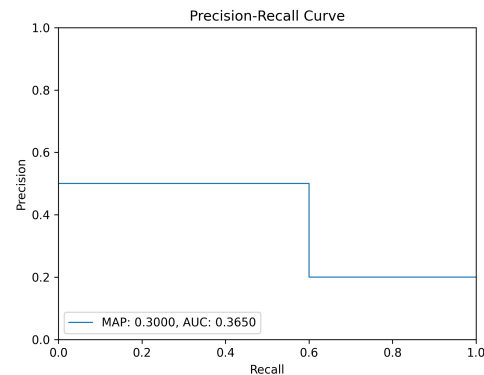


Figure 13: Precision-recall curve for query 2 in a simple system

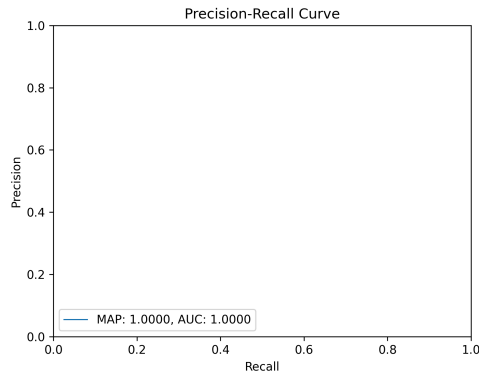


Figure 14: Precision-recall curve for query 2 in the improved system

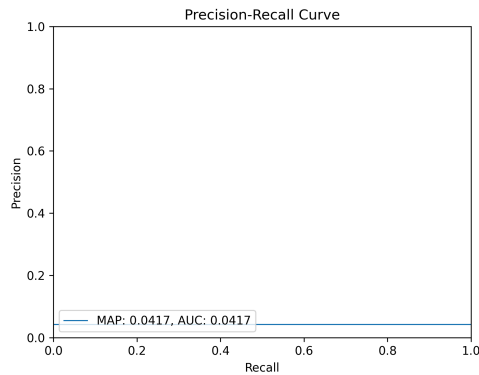


Figure 15: Precision-recall curve for query 2 in the semantic system

9.3 Query 3

Information Need: Songs whose artist is female that talk about empowerment.

This query highlights the system’s ability to retrieve songs by female artists focused on themes of empowerment, emphasizing diversity and meaningful content.

Evaluation method: To understand if the artist is female, we look at the `lastfm_tags` and also at the names of the artists. Regarding empowerment, we read the lyrics to determine if the content addresses it or includes some kind of empowering message.

Results: In this query, the results in the simple and improved systems were calculated considering only 19 values, due to the number of values the system retrieved.

By analyzing $P@10$ (Table 11), the Advanced System is much more effective at initial retrieval, ranking relevant documents with a high priority at the top of the list, with a $P@10$ of 0.8000. In contrast, the Simple System struggles significantly in ranking relevant results early, with a very low $P@10$ of 0.1000. Interestingly, Improved System demonstrates a very good outcome with a $P@10$ of 0.7000, which means that for the top-ranked documents, it also ranks relevant documents well, but not as powerfully as the Advanced System is able to.

Looking at $P@30$ in Table 11, the Advanced System yielded a high precision of 0.7333, showing consistency of relevant item retrieval across a more extensive list. The Simple System again shows an

extremely poor performance with a $P@30$ of 0.0667, hinting that its retrieved set is full of irrelevant items. The improved system strikes a balance and contains a $P@30$ value of 0.3667 while falling behind the advanced one, probably because the strength in retrieval may be more concentrated near its top-ranked results. For the semantic system, the values show that the system only retrieved a few retrieved documents.

Rank System	Simple	Advanced	Improved	Semantic
$P@10$	0.1000	0.8000	0.7000	0.1000
$P@30$	0.0667	0.7333	0.3667	0.1000
AvP	0.1534	0.8501	0.8319	0.2550

Table 11: $P@10$, $P@30$, AvP for query 3

The AvP results for this query are as follows: the Advanced System with 0.8501, followed closely by the Improved System at 0.8319, with both systems performing a good balance between precision and recall. The Semantic System has a modest 0.2550, marking an improvement over previous queries but still far behind the top-performing systems. The Simple System has, correspondingly, a struggling AvP of 0.1534, suggesting that it fails to continually retrieve relevant documents. Conclusively, the best performance is shown by Advanced and Improved Systems, and the Semantic and Simple Systems are less effective.

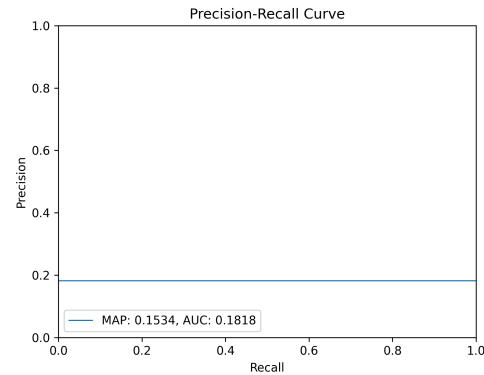


Figure 16: Precision-recall curve for query 3 in a simple system

9.4 Query 4

Information Need: Songs that talk about reflection or love.

This query explores the system’s ability to retrieve songs centered on themes of reflection or love, which are universal and deeply relevant topics.

Evaluation method: In this case, we looked for the title, `lastfm_tags`, the lyrics to see if they talk about reflection or love. Specifically for the reflection, we search for interrogative marks because it is a clue that the lyrics can speak about reflections. We also considered some references to nostalgia, childhood, past relationships, and the future.

Both the Simple and Advanced Systems are doing well in terms of $P@10$ and $P@30$, as shown in Table 12. The overall performance

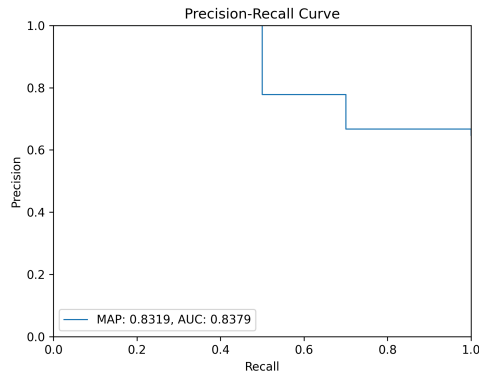


Figure 17: Precision-recall curve for query 3 in the improved system

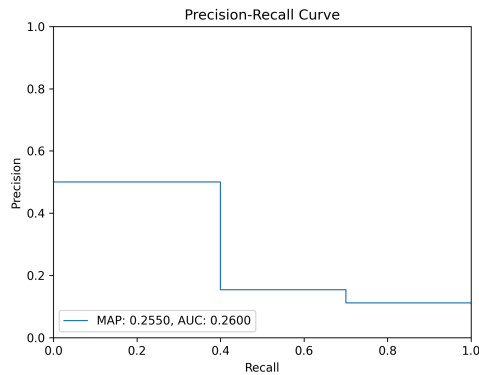


Figure 18: Precision-recall curve for query 3 in the semantic system

is slightly better for the Advanced System, though the difference is not very significant. Precisely, the Advanced System performs with a P@10 of 0.8000 against the Simple System's 0.6000, which means that out of the top 10 retrieved items, it had better precision. The improved system matches the advanced system with a P@10 of 0.8000, confirming its effectiveness in retrieving relevant documents early on. This was the best query for the semantic system, having good results for both the top 10 and the top 30. In top 10, the top results were similar to the best approach.

About P@30, the Advanced System has reached 0.7333, higher than those of both the Simple and Improved Systems, which are equal and amount to 0.6333. This reflects that although the improved system is at least competitive in early precision, it somewhat deteriorates when retrieving a larger set of documents. It may be indicative of a focused retrieval strategy that favors top-ranked items at the expense of broader recall.

Results:

Rank System	Simple	Advanced	Improved	Semantic
P@10	0.6000	0.8000	0.8000	0.7000
P@30	0.6333	0.7333	0.6333	0.5667
AvP	0.7326	0.8087	0.8301	0.6840

Table 12: P@10, P@30, AvP for query 4

The precision-recall curve for both the simple and advanced systems behaves similarly, with both performing quite well. The curve of the Advanced System (Figure 31) starts high in terms of precision and decreases gradually with the increase in recall, indicating that it retrieved strongly early and then declined a bit as more documents were considered. For the Simple System, the curve is similar in shape but with a slightly steeper decline, indicating less consistent performance.

The improved system follows a similar pattern but with higher stability for the higher recall levels, given the good early precision. Further supported by the high value of AvP of the Advanced System, this means that the system was well-balanced between precision and recall, so the Improved System is competitive, which only depends on its capability of maintaining precision across varying recall levels. This trade-off emphasizes how hard it is to have high precision and high recall in a real-world retrieval setting. The semantic system showed good results, showing almost the same results as the advanced and improved systems, even though it was just a little worse than these ones.

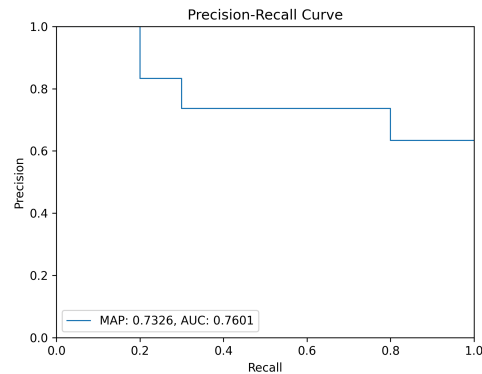


Figure 19: Precision-recall curve for query 4 in a simple system

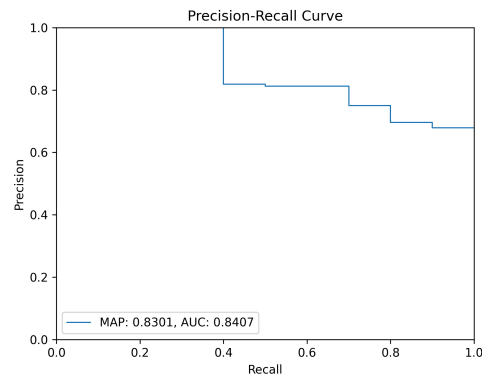


Figure 20: Precision-recall curve for query 4 in the improved system

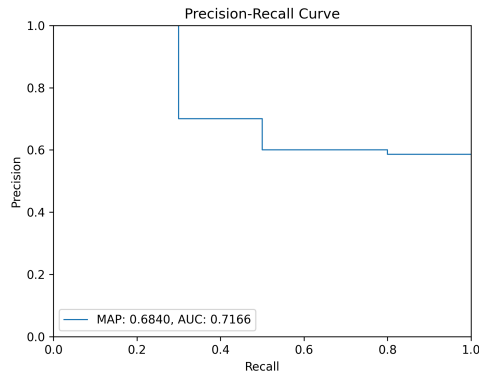


Figure 21: Precision-recall curve for query 4 in the semantic system

9.5 Query 5

Information Need: Pop songs that talk about the past.

This query focuses on retrieving pop songs with themes about the past, showcasing the system's capacity to handle genre-specific and thematic searches.

Evaluation method: To check the type of the song, we saw the lastfm_tags and the tag. To see if the content is about the past, we read the lyrics, and we tried to find out if the artist is remembering something that is appended in the past or if he misses someone or something.

Considering the P@10 values for both the Simple System and the Advanced System, the relevance of the top-ranked files is quite low, as shown in Table 13. While the Simple System has a P@10 of 0.1000, the Advanced System does moderately better with a P@10 of 0.4000. On the other hand, the improved system has a high value of 0.6000 for P@10, which shows that it retrieves more relevant files among the first 10 results.

Looking at P@30 (Table 13), the scores are higher but still relatively low for all systems. The Simple System has a P@30 of 0.2000, which is indicative of generally poor retrieval, while the Advanced System improves to 0.4667. The improved system's P@30 of 0.4333 is marginally lower than that of the advanced system, indicating a relatively stronger focus on early precision rather than performance across a larger result set. The semantic system had difficulties in retrieving relevant documents. Even the low values presented were some of the better results for this query.

This trend is reinforced by the AvP scores: the Simple System has an AvP of 0.1851, which is very poor overall in terms of retrieval effectiveness, while the Advanced System achieves 0.4216, considerably better but with significant room for improvement. The AvP for the improved system is 0.5256, which shows that the improved system performed overall better than the other systems.

Results:

Rank System	Simple	Advanced	Improved	Semantic
P@10	0.1000	0.4000	0.6000	0.2000
P@30	0.2000	0.4667	0.4333	0.1000
AvP	0.1851	0.4216	0.5256	0.4907

Table 13: P@10, P@30, AvP for query 5

The AvP results of this query are that the Improved System has the best score, 0.5256, while the Advanced System is a little behind with 0.4216, hence competitive but lower. The Semantic System performs very well, 0.4907, significantly better than in previous queries and outperforming the Advanced System. The Simple System is the weakest, with an AvP of 0.1851, reflecting its limited capabilities to retrieve relevant documents. Overall, the Improved and Semantic Systems are head and shoulders above in this query, while the Simple System trails behind.

Generally speaking, all systems show performance that is average to low, reflected in their precision-recall curves (Figures 22 and 32). Curves are mostly flat with gentle slopes followed by significant drops at higher recall. Such a pattern reflects the well-known balance between precision and recall: getting more files leads to more irrelevancies being included in the document set.

The low performance for all systems could be due to the difficulty in finding words relative to "past", as direct synonyms have multiple meanings and are usually pretty common words. However, the semantic search is also not really effective and this could be due to the query given not being enough to extract the proper semantic meaning of "past", as well as the lack of indication of what the most important columns are with boosting.

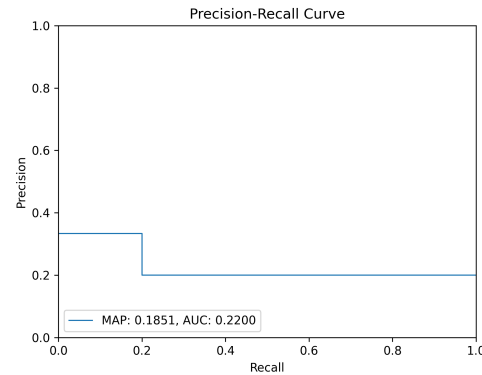


Figure 22: Precision-recall curve for query 5 in a simple system

9.6 Overall System Evaluation

Systems:

Global System	Simple	Advanced	Improved	Semantic
MAP	0.3848	0.7632	0.7869	0.3026

Table 14: MAP averages for both systems

Based on these averages, we conclude that using the improved schema and boosted queries produced the best performance for the top 10 results (P@10), being slightly better than the advanced schema with the boosted queries. The improved system yielded better precision in the top 10 results but worse precision in the top 30 results, which means that these two systems have a trade-off between having better results at the top 10 (improved system) or better top 30 (advanced system). Our choice for the best system

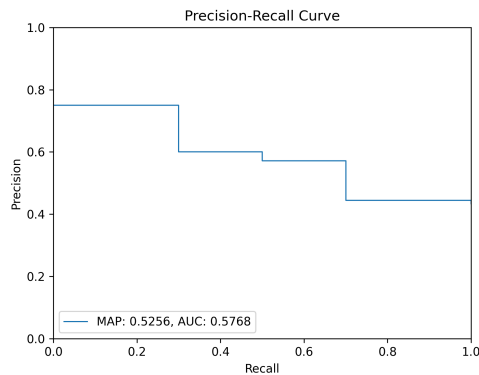


Figure 23: Precision-recall curve for query 5 in the improved system

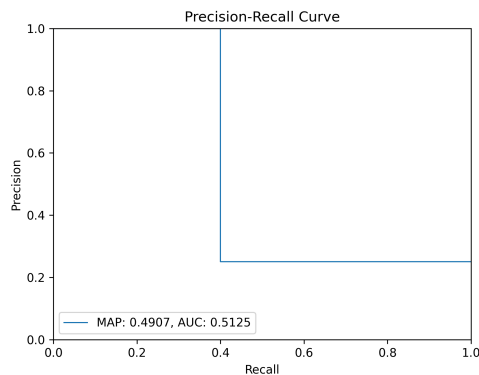


Figure 24: Precision-recall curve for query 5 in the semantic system

would be the improved one due to the fact that it still presents a little higher MAP, and normally the top 10 results are the ones that matter the most to the user of the search system.

Contrary to what we expected, the semantic scheme produced the worst results. We consider that these results come because, in the other schemas, we put more weight on some parameters that are relevant to our information needs. For example, we took into account the tags from `last_fm` that give us important information in some queries, like whether the singer is female or not or if the singer is Portuguese. Another reason is that some lyrics, which are our main parameter to do a full text search, are short. This increases the difficulty to find good matches. To finish with, the Portuguese language is full of different verb conjugations that make the task more difficult, and there are only a few models available to train in Portuguese.

Some of the queries have text in English because having text in Portuguese gave us almost no results. For example, in query 2, we needed to replace "número 1" with "number 1" because the option with Portuguese text gave us less than ten results, each of which didn't really allow us to evaluate the query result efficiently. With the replacement of the Portuguese text with an English one, we got over 50 results, and that granted us a better evaluation because we got a lot more documents to evaluate.

10 Conclusion

In the end, we have a solid basis for a retrieval system that allows users to easily find their desired music/music type based on theme or even music characteristics. This sets a clear base on which we can then expand the project to achieve what ultimately led us to start it in the first place: helping other people find new and old favorite songs. Of course, in this work, we can detect some flaws in our system, which we will focus on enhancing in future work.

These steps were quite challenging overall, since finding the right dataset, deciding which part of the dataset would be used while keeping a substantial enough size, and uniting the prepared dataset with the API used to obtain extra information were all challenging. However, with a good preparation of the data, we overcame these difficulties. Also, the lack of filters for Portuguese in Solr was quite challenging, as we had to adapt to the existing filters and work with a small set.

It was quite challenging to also use the semantics for this system, as it took a while to find the right model and the results weren't the most desired.

After all this work, we learned a lot about the search retrieval area and gained a lot of interest into developing this project furtherly.

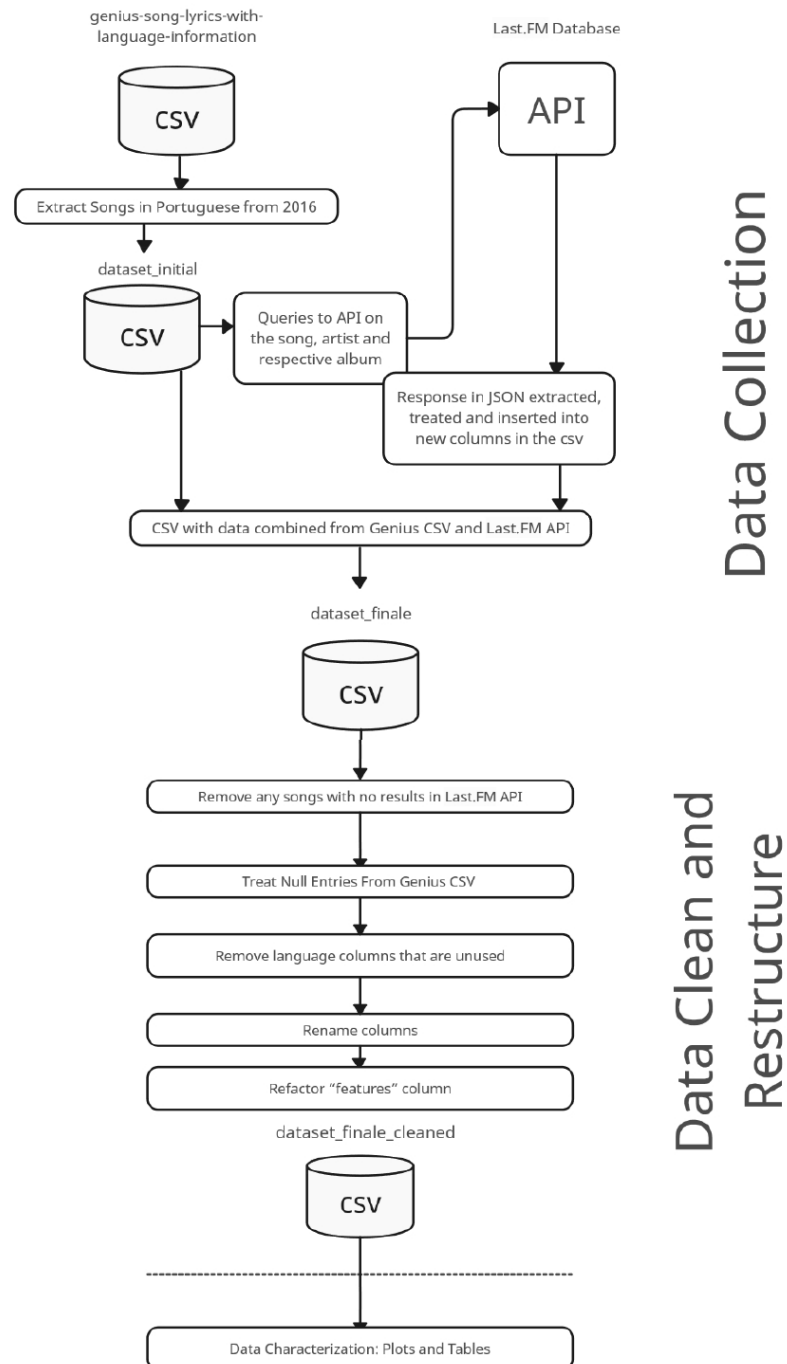
References

- [1] Steven Bird, Edward Loper, and Ewan Klein. 2023. Natural Language Toolkit. <https://www.nltk.org/> Version 3.6.7.
- [2] Dremio Corporation. 2024. *Dremio: The Easy and Open Data Lakehouse*. <https://www.dremio.com/> Accessed: 2024-11-21.
- [3] Carlos Garcia. 2022. Genius Song Lyrics with Language Information. <https://www.kaggle.com/datasets/carlosgcdj/genius-song-lyrics-with-language-information>. Accessed: 2024-10-13.
- [4] Last.fm. 2024. Last.fm API. <https://www.last.fm/api>. Accessed: 2024-10-13.
- [5] Andreas Mueller. 2018. wordcloud: A little word cloud generator in Python. https://github.com/amueller/word_cloud
- [6] Solr. 2024. Solr. <https://solr.apache.org/>. Accessed: 2024-11-15.
- [7] Matplotlib Development Team. 2024. Matplotlib Documentation. <https://matplotlib.org/>. Accessed: 2024-10-13.
- [8] NumPy Development Team. 2024. NumPy Documentation. <https://numpy.org/>. Accessed: 2024-10-13.
- [9] Pandas Development Team. 2024. Pandas Documentation. <https://pandas.pydata.org/>. Accessed: 2024-10-13.

[1–9].

A Annexes

Revised 19 December 2024

**Figure 26: Data Pipeline**

Song Search System

Non-Semantic

pop e passada

Search

Results:

root: 2668232c-839c-4af5-b281-59fd4436550b

album: Outras Histórias

artist: Deolinda

features: {}

lastfm_content_artist: Deolinda is a project from Lisbon, Portugal, inspired by fado and its traditional roots in order to create original songs based on portuguese traditional and popular music.

It came to life in 2006, by the hands of 4 young musicians with diverse musical backgrounds and experiences (such as classical music, jazz, ethnical and traditional music), searching to recreate a sound rooted in popular music through the crossing of different musical languages and musical research serving as a base for the group's original compositions. Their first album - Canção ao Lado was released in 2008.

" Her name is Deolinda and she is old enough to realize that life isnt as easy as it seems. Merrily unmarried, in love and out of love, born in Lisbon, she inhabits a ground floor apartment somewhere in the suburbs of the capital. She writes her own songs by peeking through the curtains of her window, drawing inspiration from the old gramophone records that once belonged to her grandmother and by the bizarre and strange life of her neighbours. She lives with 2 cats and a gold fish. "

Deolinda are: Ana Bacalhau (vocals), Zé Pedro Leitão (doublebass), Pedro da Silva Martins (classical guitar) and Luís José Martins (classical guitar). Read more on Last fm. User contributed text is available under the Creative Commons By SA License, additional terms may apply

lastfm_different_listeners: 733

lastfm_image_album: https://lastfm.freetls.fastly.net/u/300x300/2e601d9b54b43a2e94dc5cd920419be.jpg

lastfm_image_artist: https://lastfm.freetls.fastly.net/u/300x300/2a96cbd8b46e442f641c2b6b621562f.png

lastfm_playcount: 2459

Figure 27: Interface of the system.

Number	System Simple	System Advanced	System Improved	System Semantic
1	R	R	R	N
2	N	R	R	N
3	N	R	R	N
4	N	R	R	N
5	N	R	R	N
6	N	N	N	N
7	R	N	N	N
8	N	N	R	N
9	R	R	N	N
10	R	N	R	N

Table 16: Top 10 relevant retrieved documents for query 1

Number	System Simple	System Advanced	System Improved	System Semantic
1	N	R	R	N
2	N	R	R	N
3	R	R	R	N
4	R	R	R	N
5	N	R	R	N
6	N	R	R	N
7	N	R	R	N
8	N	R	R	N
9	N	R	R	N
10	N	R	R	N

Table 17: Top 10 relevant retrieved documents for query 2

Number	System Simple	System Advanced	System Improved	System Semantic
1	N	R	R	N
2	N	R	R	R
3	N	R	R	N
4	N	R	R	N
5	N	R	R	N
6	N	N	N	N
7	N	R	N	N
8	R	R	R	N
9	N	R	R	N
10	N	N	N	N

Table 18: Top 10 relevant retrieved documents for query 3

Number	System Simple	System Advanced	System Improved	System Semantic
1	R	R	R	R
2	R	R	R	R
3	R	R	R	R
4	N	R	R	R
5	R	R	R	N
6	R	N	R	N
7	N	R	N	N
8	N	N	N	R
9	N	R	R	R
10	R	R	R	R

Table 19: Top 10 relevant retrieved documents for query 4

Number	System Simple	System Advanced	System Improved	System Semantic
1	N	N	N	R
2	R	N	R	N
3	N	R	R	N
4	N	N	R	N
5	N	R	N	N
6	N	R	N	N
7	N	N	N	N
8	N	N	R	N
9	N	R	R	R
10	N	N	R	N

Table 20: Top 10 relevant retrieved documents for query 5

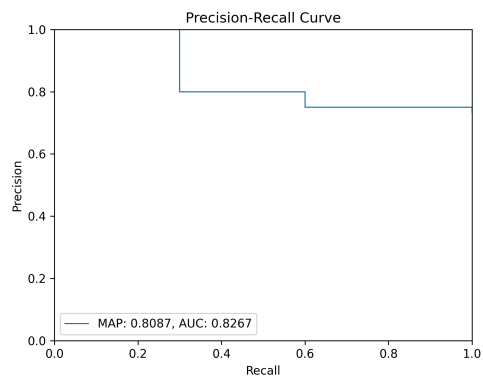


Figure 31: Precision-recall curve for query 4 in advanced system

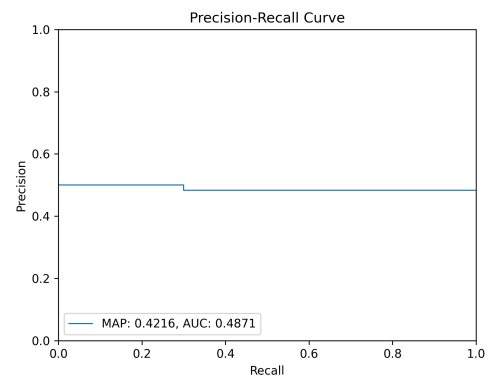


Figure 32: Precision-recall curve for query 5 in advanced system